

**FUNDAÇÃO ESCOLA DE COMÉRCIO ÁLVARES PENTEADO
FECAP**

**CENTRO UNIVERSITÁRIO ÁLVARES PENTEADO
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS**

Cauã Santos Nascimento

Pedro Henrique Santos da Silva

Paulo Miguel Miranda Marcelli

Gabriel Vaz Ferreira Neves

**PROJETO INTEGRADO
ALGORITMOS E LÓGICA DE PROGRAMAÇÃO**

PROFESSOR(A): LUCY MARI TABUTI

São Paulo

2026

Sumário

1. Introdução.....	3
2. Estrutura de Modularização	4
2.1 Entrada e Saída.....	4
3. Estrutura de Vetores.....	5
3.1 Entrada e Saída.....	5

1. Introdução

Este documento apresenta a implementação de três estruturas fundamentais da programação, modularização, vetores e matrizes aplicadas ao projeto SchoolPlayManager, desenvolvido em C# com Windows Forms. O sistema gerencia assinaturas de jogos educacionais por escolas, controlando pacotes contratados, limites de acessos mensais, IPs autorizados e relatórios de consumo. As estruturas apresentadas ampliam as funcionalidades do sistema, organizando os dados, separando responsabilidades e registrando resultados.

2. Estrutura de Modularização

A estrutura de modularização é uma técnica de organização que divide o programa em módulos independentes, cada um com uma função específica. Dentro do projeto, a classe AcessoManager coordena o fluxo completo de acesso ao sistema chamando métodos especializados em sequência: validação de IP, verificação do limite mensal e registro do acesso.

2.1 Entrada e saída

Entrada:

ProcessarAcesso(): IP de origem; nomeEscola: nome da escola; nomeJogo: nome do jogo.

Saída:

Valor booleano indicando se o acesso foi concedido(True) ou negado(false).

Trecho do código – Classe AcessoManager:

```
// MODULARIZAÇÃO
0 referências
public class AcessoManager
{
    private IPValidator _ipValidator = new IPValidator();
    private List<string> _logDeAcessos = new List<string>();
    private int _consumoMensal = 0;
    private const int LIMITE_MENSAL = 100;

    //coordena todos os módulos de validação e registro
    0 referências
    public bool ProcessarAcesso(string ipTentativa, string nomeEscola, string nomeJogo)
    {
        bool ipValido = ValidarIP(ipTentativa);
        bool dentroLimite = VerificarLimiteMensal();

        if (ipValido && dentroLimite)
        {
            RegistrarAcesso(nomeEscola, nomeJogo);
            return true;
        }

        return false;
    }

    //delega a validação do IP
    1 referência
    private bool ValidarIP(string ip)
    {
        return _ipValidator.ValidarAcesso(ip);
    }

    //Verifica se o consumo permanece dentro do limite
    1 referência
    private bool VerificarLimiteMensal()
    {
        return _consumoMensal < LIMITE_MENSAL;
    }

    // Incrementa o consumo e registra o acesso
    1 referência
    private void RegistrarAcesso(string nomeEscola, string nomeJogo)
    {
        _consumoMensal++;
        string registro = $"{DateTime.Now:dd/MM/yyyy HH:mm} | {nomeEscola} | {nomeJogo}";
        _logDeAcessos.Add(registro);
    }

    //retorna o consumo atual
    0 referências
    public int ConsultarConsumoAtual()
    {
        return _consumoMensal;
    }

    // retorna os registros de log
    0 referências
    public List<string> ObterLog()
    {
        return _logDeAcessos;
    }
}
```

3. Estrutura de Vetores e Matrizes

A estrutura de vetores e matrizes armazenam múltiplos valores do mesmo tipo em posições indexadas. Um vetor é unidimensional, ele é uma sequência linear de valores acessados por um único índice. Uma matriz é bidimensional, ela é uma tabela de linhas e colunas acessadas por dois índices. Dentro do projeto, as duas estruturas são utilizadas em conjunto na classe PermissaoJogos, um vetor armazena os nomes dos jogos disponíveis na plataforma, e uma matriz booleana registra quais jogos cada escola tem permissão de acessar conforme o pacote contratado.

3.1 Entrada e Saída

Entrada:

Índices da escola (indexEscola) e do jogo (indexJogo), passados ao método VerificarPermissao() quando a escola tenta acessar um jogo do catálogo.

Saída:

Um valor booleano, true se a escola tiver permissão para acessar ao jogo, ou false caso o jogo não esteja incluso no pacote contratado. O método ListarJogosPermitidos() retorna uma string com os jogos liberados.

Trecho do código – Classe PermissaoJogos:

```
//VETORES E MATRIZES
1 referência
public class PermissaoJogos
{
    //Nomes das escolas
    private string[] _escolas = { "Escola A", "Escola B", "Escola C" };

    //Nomes dos jogos
    private string[] _jogos = { "Matemática", "Ciências", "História" };

    //Matriz escola [escola, jogo] = verificar a permissao da escola
    private bool[,] _matrizPermissoes;

    0 referências
    public PermissaoJogos()
    {
        _matrizPermissoes = new bool[_escolas.Length, _jogos.Length]
        {
            { true, true, false, false },
            { true, true, true, true },
            { true, false, false, false }
        };
    }

    // Consulta a célula [escola, jogo] para verificar a permissão
    0 referências
    public bool VerificarPermissao(int indexEscola, int indexJogos)
    {
        return _matrizPermissoes[indexEscola, indexJogos];
    }

    //Percorre o vetor de jogos e a linha da escola na matriz para listar permissões
    0 referências
    public string ListarJogosPermitidos(int indexEscola)
    {
        string resultado = $"jogos disponíveis para {_escolas[indexEscola]}:\n";

        for (int j = 0; j < _jogos.Length; j++)
        {
            if (_matrizPermissoes[indexEscola, j])
            {
                resultado += $" - {_jogos[j]}\n";
            }
        }

        return resultado;
    }
}
```